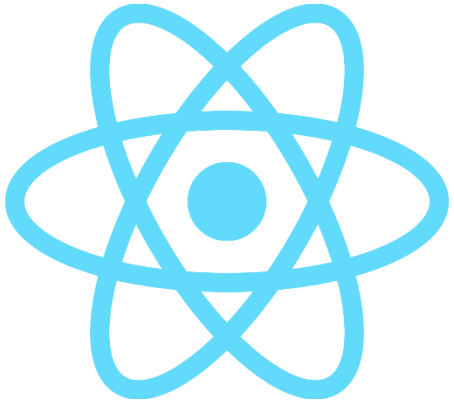
React JS



Introduction to ReactJS

- ReactJS is an open-source JavaScript library created by Facebook's Jordan Walke to make user interfaces for both web and mobile systems.
- React was first used in 2011 on Facebook's newsfeed. In 2012, it was used on Instagram, and in 2013 it was made available to the public.
- React is best for applications that change their data quickly and dynamically.
- It can show the parts of the UI that are changing without re-rendering the whole page. This makes the user experience much faster.

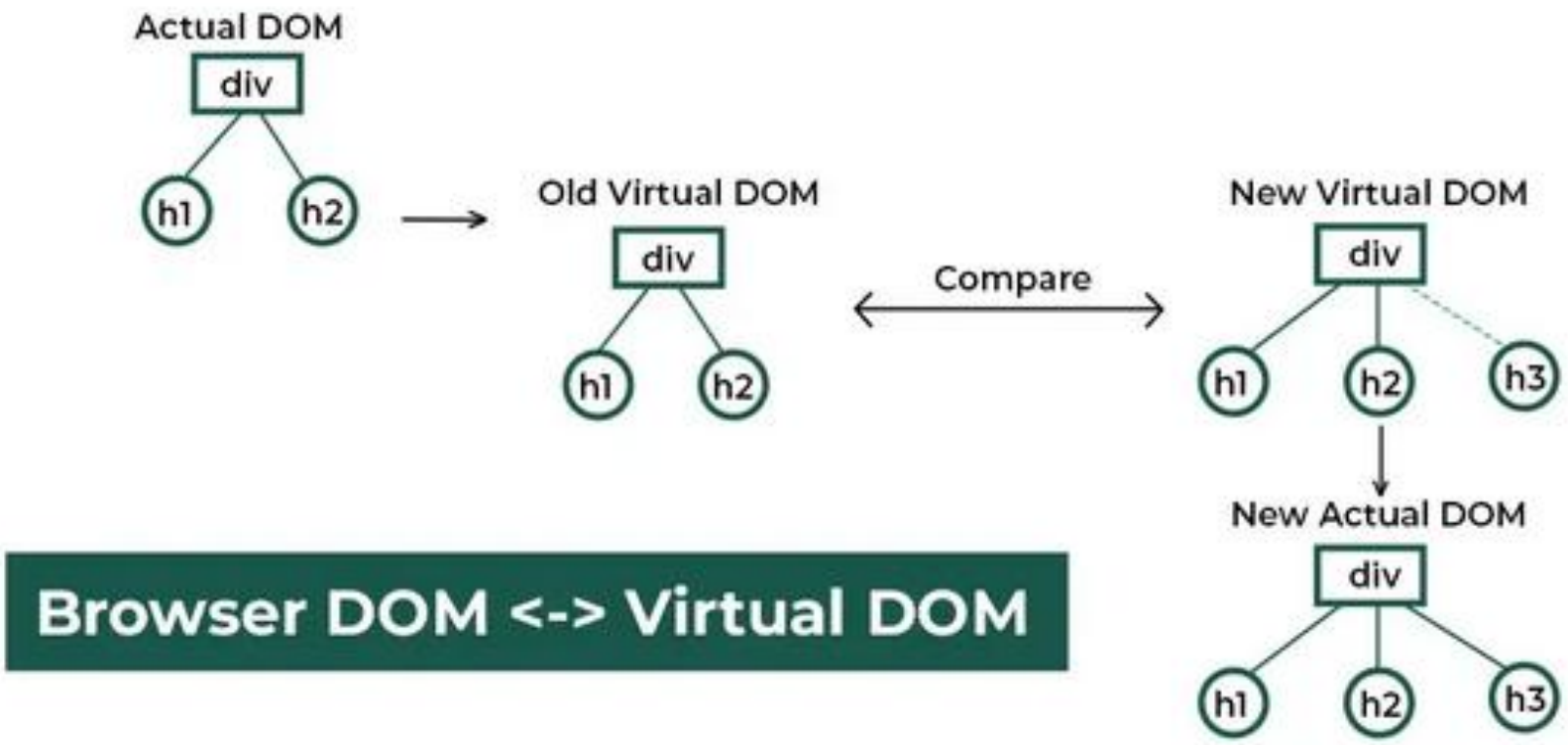
Why React?

- It simplifies the creation of single-page applications (SPAs) with a focus on performance and maintainability.
- Easy creation of dynamic applications: React makes it easier to create dynamic web applications because it requires less coding.
- Reusable components: Components are the building blocks of any React application, and a single app usually consists of multiple components. These components have their logic and controls, and they can be reused throughout the application

Why React?

- Unidirectional data flow: React follows a unidirectional data flow. This means that when designing a React app, developers often nest child components within parent components.
- Small learning curve: React is easy to learn, as it mostly combines basic HTML and JavaScript concepts with some beneficial additions.
- Dedicated tools for easy debugging: Facebook has released a Chrome extension that can be used to debug React applications.

Virtual Document Object Model (DOM)



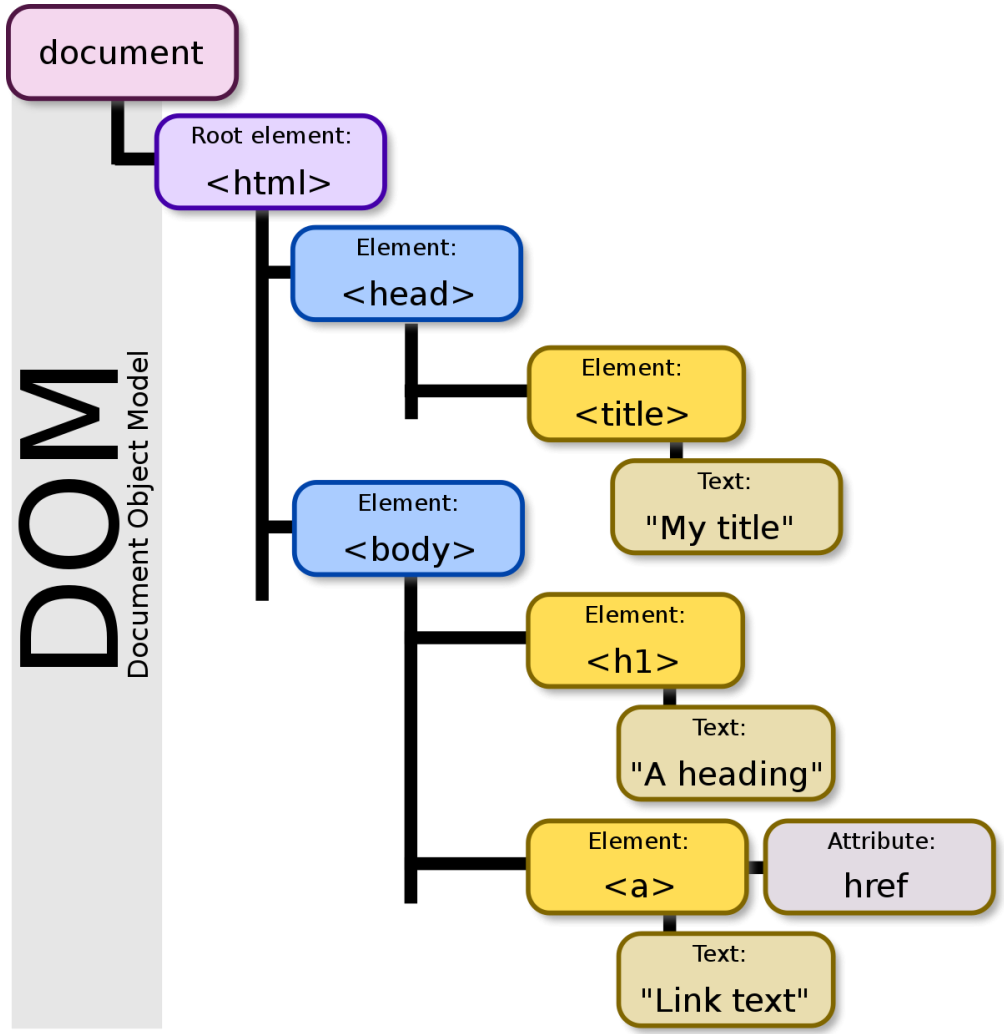
React compares the previous Virtual DOM with the New Virtual DOM using a process called reconciliation.

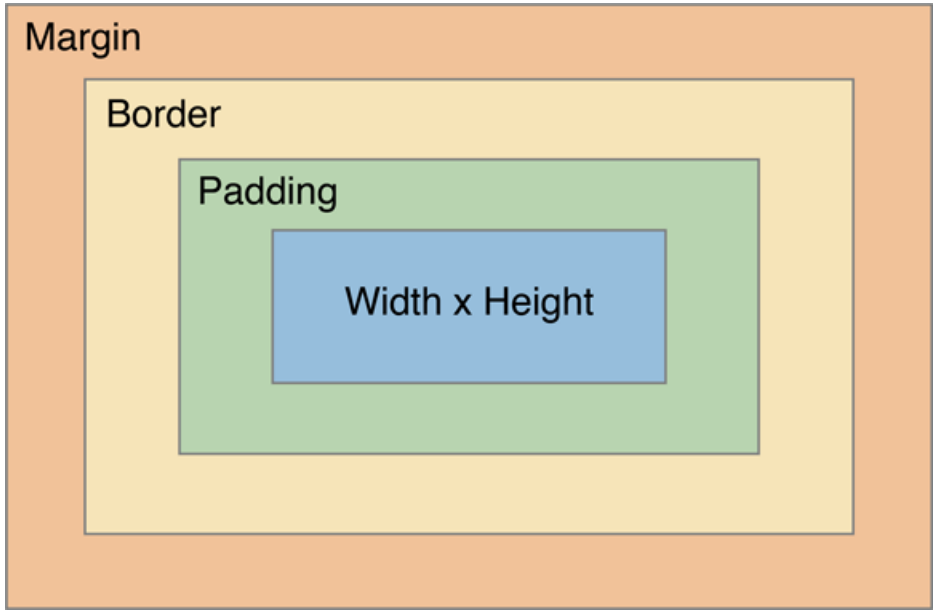
JSX - Javascript syntax extensions

- JSX is a syntax extension to JavaScript
- By using JSX, you can write the HTML structures in the same file that contain Javascript codes

```
// JSX

const example1 = <p id='large'>hello</p>;
const example2 = <p id='small'>hi</p>;
```



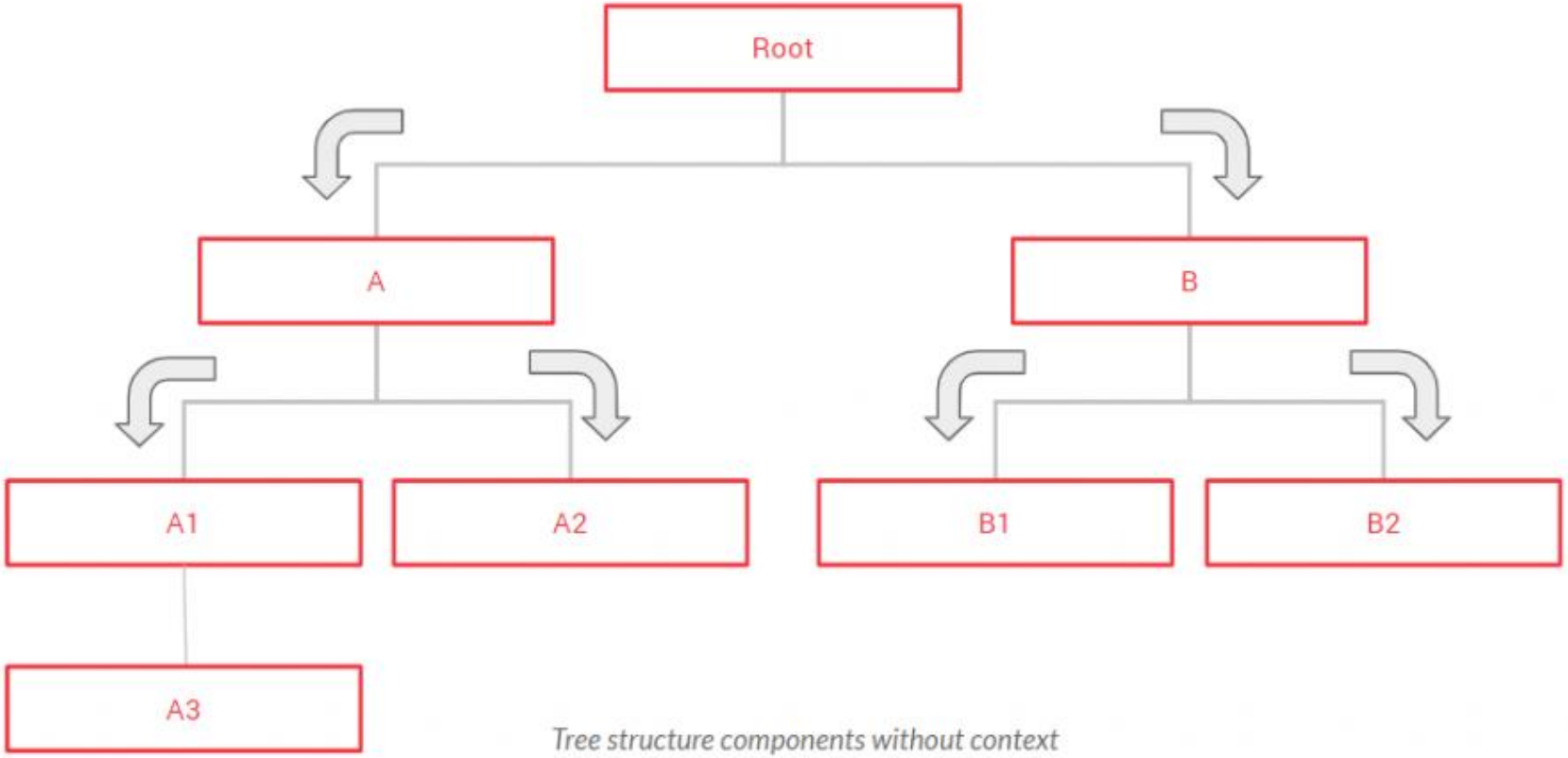


CSS Box Model

Architecture

- Model View Controller (MVC) is an architectural pattern that splits the application layer into Model, View, and Controller.
- The model relates to all data-related logic; the view is used for the UI logic of the application, and the controller is an interface between the Model and View.
- React is the 'View' responsible for how the app looks and feels.

React Architecture



Tree structure components without context

Installation

https://reactjs.org/docs/create-a-new-react-app.html

- Install Node.js with default settings
- Run the command **npx create-react-app appname**
- Change directory to appname **cd appname**
- Run the command **npm start**
- Enjoy!

React Components

Types of components

Components

Stateless Functional Components

They are JavaScript functions

They return HTML

Can be contained in a .js or .jsx file

Stateful Class Component

Regular classes that **extend** the **component** class

They must contain a **render** method that returns HTML

Can be contained in a .js or .jsx file

Components are the **building blocks** of a React application that represent **a part of the user interface**

Render method

In its minimal form, a component must define a render method that specifies how the component renders to the DOM

Passing properties

A component can also receive props. These are properties passed by its parent to specify values

Components are the **building blocks** of a React application that represent **a part of the user interface**

Re-usability

A component used in one area of the application can be reused in another area. This helps speed up the development process

Nested Components

A component can contain several other components

Features

- React components are independent, reusable building blocks in a React application that define what gets displayed on the UI.
- They accept inputs called props and return React elements describing the UI.

Features

- Components can be reused across different parts of the application to maintain consistency and reduce code duplication.
- Components manage how their output is rendered in the DOM based on their state and props.
- React loads only the necessary components, ensuring optimized performance.
- Only the specific Component updates instead of the whole page.

Functional Component

- Functional components are simply javascript functions. We can create a functional component in React by writing a javascript function. These functions may or may not receive data as parameters
- we can compose multiple functional components under a single functional component

Example

```
function Greet(props) {
  return <h1>Hello, {props.name}!</h1>;
}
```

Class Components

- The class components are a little more complex than the functional components
- On a demise, not used on a large scale.

Class Components

- Class components are ES6 classes that extend React.Component.
- They include additional features like state management and lifecycle methods.

- State Management: State is managed using the this.state property.
- Lifecycle Methods: Includes methods like componentDidMount, componentDidUpdate, etc.

Example

```
class Greet extends React.Component {
  render() {
   return <h1>Hello, {this.props.name}!</h1>;
}
}
```

File Edit Selection View Go Run Terminal Help

react-app

EXPLORER

REACT-APP

- node_modules
- public
- src
  - assets
  - components
    - header.jsx
    - hello.jsx
  - App.css
  - App.tsx 2
  - index.css
  - main.tsx
  - Message.tsx
  - mycomponent.jsx
  - vite-env.d.ts
- .gitignore
- eslint.config.js
- index.html
- package-lock.json
- package.json
- README.md
- tsconfig.app.json
- tsconfig.json
- tsconfig.node.json
- vite.config.ts

OUTLINE

TIMELINE

Message.tsx | hello.jsx | header.jsx | App.css | App.tsx 2 | mycomponent.jsx | index.css

src > components > hello.jsx > Hello

```jsx
export default function Hello() {
const name="Amruta";
return(
    <>
    <h1> Hello . {name}</h1>

    </>
);
}
```

PROBLEMS 2 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

```
11:10:05 am [vite] (client) hmr update /src/components/hello.jsx (x4)
```

powershell
cmd
esbuild

File Edit Selection View Go Run Terminal Help

react-app

EXPLORER

REACT-APP

- node_modules
- public
- src
  - assets
  - components
    - header.jsx
    - hello.jsx
  - App.css
  - App.tsx 2
  - index.css
  - main.tsx
  - Message.tsx
  - mycomponent.jsx
  - vite-env.d.ts
- .gitignore
- eslint.config.js
- index.html
- package-lock.json
- package.json
- README.md
- tsconfig.app.json
- tsconfig.json
- tsconfig.node.json
- vite.config.ts

OUTLINE

TIMELINE

Message.tsx | hello.jsx | header.jsx | App.css | App.tsx 2 | mycomponent.jsx | index.css

src > components > header.jsx > Header

```jsx
export default function Header(){

return(
    <div>
    <h1>   Header Component</h1>
    </div>
);

}
```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
11:10:05 am [vite] (client) hmr update /src/components/hello.jsx (x4)
```

powershell
cmd
esbuild

File Edit Selection View Go Run Terminal Help

react-app

EXPLORER

REACT-APP

- node_modules
- public
- src
  - assets
  - components
    - header.jsx
    - hello.jsx
  - App.css
  - App.tsx 2
  - index.css
  - main.tsx
  - Message.tsx
  - mycomponent.jsx
  - vite-env.d.ts
- .gitignore
- eslint.config.js
- index.html
- package-lock.json
- package.json
- README.md
- tsconfig.app.json
- tsconfig.json
- tsconfig.node.json
- vite.config.ts

OUTLINE

TIMELINE

Message.tsx | hello.jsx | header.jsx | App.css | App.tsx 2 | mycomponent.jsx | index.css

src > App.tsx > default

```
import Hello from './components/hello';
import Header from './components/header';


function App() {
  return (
    <div>
      <Header></Header>
      <Hello></Hello>
        </div>
  );
}

export default App;
```

PROBLEMS 2 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

```
11:10:05 am [vite] (client) hmr update /src/components/hello.jsx (x4)
```

powershell
cmd
esbuild

Vite + React + TS

localhost:5173

Header Component

Updated Header Component

Nav bar

Hello . Amruta

File Edit Selection View Go Run Terminal Help

react-app

Message.tsx | hello.jsx | header.jsx | funcarrow.jsx | App.css | App.tsx 3 | mycomponent.jsx | index.css

src > components > funcarrow.jsx > default

```
const NHeader=() =>{

    return(
        <div>
        <h2> Updated Header Component</h2>
        </div>
    );

    }
    export default NHeader;
```

PROBLEMS 3 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

```
11:24:49 am [vite] (client) hmr update /src/App.tsx (x2)
```

powershell
cmd
esbuild

File Edit Selection View Go Run Terminal Help

react-app

EXPLORER

REACT-APP

- node_modules
- public
- src
  - assets
  - components
    - funcarrow.jsx
    - header.jsx
    - hello.jsx
    - navbar.jsx
  - App.css
  - App.tsx 4
  - index.css
  - main.tsx
  - Message.tsx
  - mycomponent.jsx
  - vite-env.d.ts
- .gitignore
- eslint.config.js
- index.html
- package-lock.json
- package.json
- README.md
- tsconfig.app.json
- tsconfig.json
- tsconfig.node.json
- vite.config.ts

OUTLINE

Message.tsx | hello.jsx | header.jsx | funcarrow.jsx | navbar.jsx | App.css | App.tsx 4 | mycomponent.jsx | index.css

src > components > navbar.jsx > default

```jsx
import React from "react";

class Navbar extends  React.Component{
    render()  {
        return <h1>Nav bar</h1>
    }
}

export default Navbar;
```

PROBLEMS 4 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

```
11:42:37 am [vite] (client) hmr update /src/App.tsx
```

powershell
cmd
esbuild

File Edit Selection View Go Run Terminal Help

react-app

EXPLORER

REACT-APP

- node_modules
- public
- src
  - assets
  - components
    - funcarrow.jsx
    - header.jsx
    - hello.jsx
    - navbar.jsx
  - App.css
  - App.tsx 4
  - index.css
  - main.tsx
  - Message.tsx
  - mycomponent.jsx
  - vite-env.d.ts
- .gitignore
- eslint.config.js
- index.html
- package-lock.json
- package.json
- README.md
- tsconfig.app.json
- tsconfig.json
- tsconfig.node.json
- vite.config.ts

OUTLINE

TIMELINE

Message.tsx | hello.jsx | header.jsx | funcarrow.jsx | navbar.jsx | App.css | App.tsx 4 | mycomponent.jsx | index.css

src > App.tsx > App

```tsx
import Hello from './components/hello';
import Header from './components/header';
import NHeader from './components/funcarrow';
import Navbar from './components/navbar';



function App() {
  return (
    <div>
      <Header></Header>
      <NHeader></NHeader>
      <Navbar></Navbar>
      <Hello></Hello>
        </div>
  );
}

export default App;
```

PROBLEMS 4 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

```
11:42:37 am [vite] (client) hmr update /src/App.tsx
```

- powershell
- cmd
- esbuild

Function Components vs Class Components

Function Components	Class Components
Written as JavaScript functions	Written as ES6 classes that extend the React.Component class
Simpler and more concise	Little Complex than function Components
Uses React hooks for state management	State management is handled using ‘this.state’
Preferred for most use cases in modern react	Still used in older React

State

A state is an object that **stores the values** of properties belonging to a component that could change over a period of time

A state can be modified based on the user action or network changes

The state object can store **multiple properties**

Every time the state of an object changes, React re-renders the component to the browser

`this.setState()` is used to change the value of the state object

The state object is initialized in the constructor

`setState()` function performs a shallow merge between the new and the previous state

State Management in React

- ‘useState()’ -for function components
- ‘setState()’ -with ‘this.state’ for class components

File Edit Selection View Go Run Terminal Help

react-app

EXPLORER

REACT-APP

- node_modules
- public
- src
  - assets
  - components
    - counter.jsx
    - funcarrow.jsx
    - header.jsx
    - hello.jsx
    - navbar.jsx
  - App.css
  - App.tsx 5
  - index.css
  - main.tsx
  - Message.tsx
  - mycomponent.jsx
  - vite-env.d.ts
- .gitignore
- eslint.config.js
- index.html
- package-lock.json
- package.json
- README.md
- tsconfig.app.json
- tsconfig.json
- tsconfig.node.json
- vite.config.ts

Message.tsx | hello.jsx | header.jsx | funcarrow.jsx | navbar.jsx | App.css | App.tsx 5 | counter.jsx

src > components > counter.jsx > Counter

```
import {useState}from "react"
export default function Counter(){
const[count, setCount]=useState(0);
function increment(){
    setCount((prevValue)=>prevValue+1);
}

    return(
        <div>
            <p> Count: .{count}</p>
            <button onClick={increment}> Click to increment </button>

        </div>
    );

    }
```

PROBLEMS 5 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

```
3:26:36 pm [vite] (client) hmr update /src/components/counter.jsx (x4)
```

File Edit Selection View Go Run Terminal Help

react-app

EXPLORER

REACT-APP

- node_modules
- public
- src
  - assets
  - components
    - counter.jsx
    - funcarrow.jsx
    - header.jsx
    - hello.jsx
    - navbar.jsx
  - App.css
  - App.tsx 5
  - index.css
  - main.tsx
  - Message.tsx
  - mycomponent.jsx
  - vite-env.d.ts
- .gitignore
- eslint.config.js
- index.html
- package-lock.json
- package.json
- README.md
- tsconfig.app.json
- tsconfig.json
- tsconfig.node.json
- vite.config.ts

Message.tsx | hello.jsx | header.jsx | funcarrow.jsx | navbar.jsx | App.css | App.tsx 5 | counter.jsx

src > components > counter.jsx > Counter

```jsx
import {useState}from "react"
export default function Counter(){
const[count, setCount]=useState(0);
function increment(){
    setCount(count+1);
}
    return(
        <div>
            <p> Count: .{count}</p>
            <button onClick={increment}> Click to increment </button>

        </div>
    );

    }
```

PROBLEMS 5 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

```
3:20:40 pm [vite] (client) hmr update /src/components/counter.jsx
```

Vite + React + TS

localhost:5173

Header Component

Updated Header Component

Nav bar

Hello . Amruta

Count: .2

Click to increment

File Edit Selection View Go Run Terminal Help react-app

EXPLORER

REACT-APP
- node_modules
- public
- src
  - assets
  - components
    - counter.jsx
    - funcarrow.jsx
    - header.jsx
    - hello.jsx
    - navbar.jsx
  - App.css
  - App.tsx 5
  - index.css
  - main.tsx
  - Message.tsx
  - mycomponent.jsx
  - vite-env.d.ts
- .gitignore
- eslint.config.js
- index.html
- package-lock.json
- package.json
- README.md
- tsconfig.app.json
- tsconfig.json
- tsconfig.node.json
- vite.config.ts

Message.tsx | hello.jsx | header.jsx | funcarrow.jsx | navbar.jsx | App.css | App.tsx 5 | counter.jsx

src > components > counter.jsx > Counter > incrementfive

```jsx
import {useState}from "react"
export default function Counter(){
const[count, setCount]=useState(0);
function increment(){
    setCount((prevValue)=>prevValue+1);
}
function incrementfive(){
    increment();
    increment();
    increment();
    increment();
    increment();
}

    return(
        <div>
            <p> Count: .{count}</p>
            <button onClick={increment}> Click to increment </button>
            <button onClick={incrementfive}> Click to increment by 5 </button>

        </div>
    );

    }
```

Vite + React + TS

localhost:5173

Header Component

Updated Header Component

Nav bar

Hello . Amruta

Count: .20

Click to increment | Click to increment by 5

To set state using class component

- Write a code to add ItemList component and set state to add new item in the list on button click.

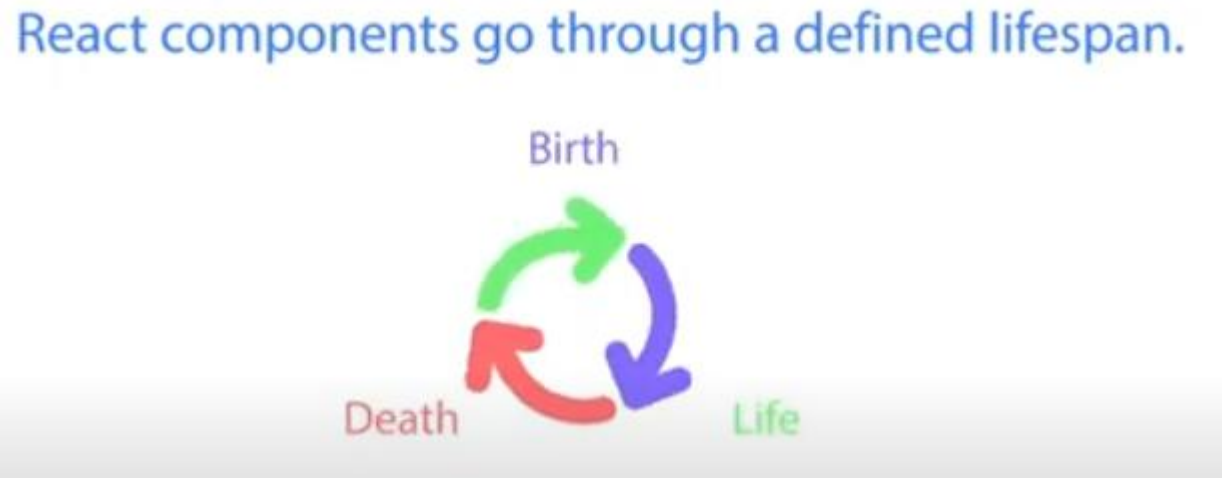
React Lifecycle

- In React, the lifecycle refers to the various stages a component goes through.
- These stages allow developers to run specific code at key moments, such as when the component is created, updated, or removed.

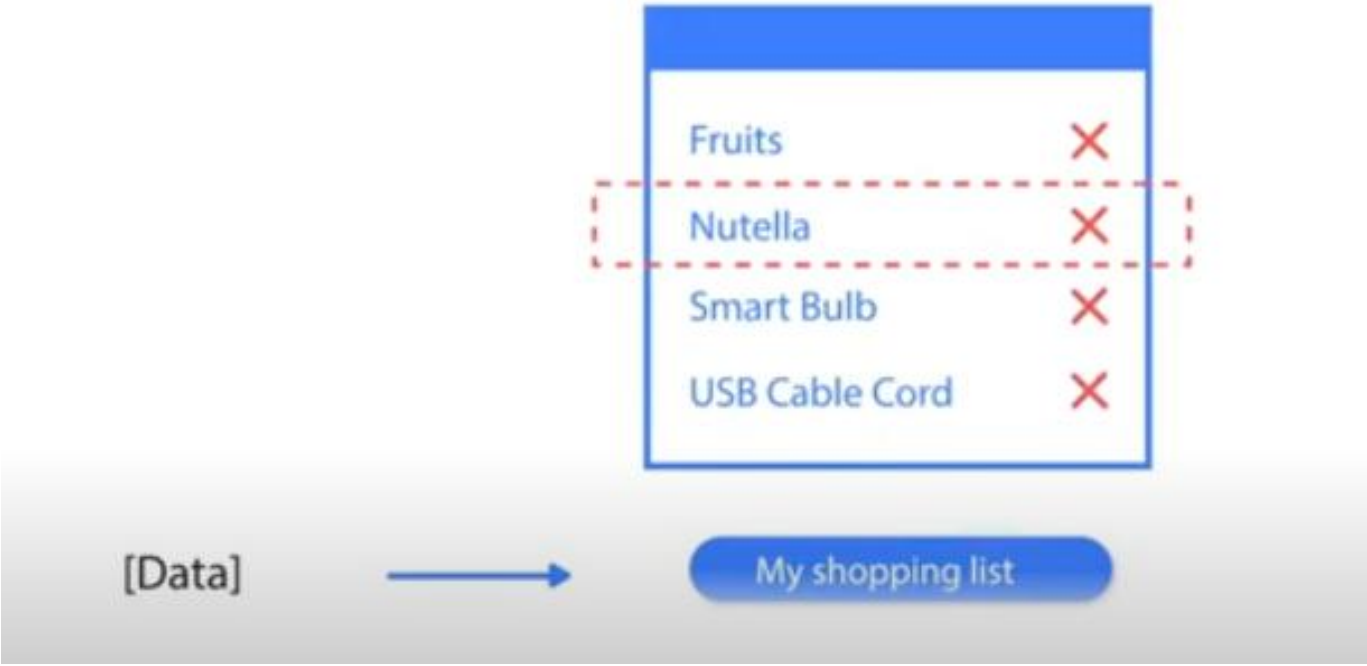
Lifecycle of a Component

Introduction

- A React application is made of several components.

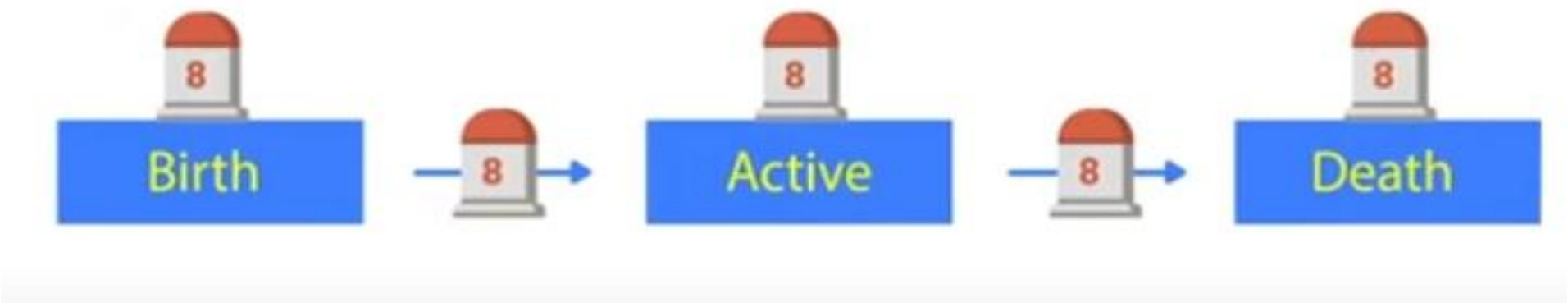


List item component instances are brought to life when the button is clicked

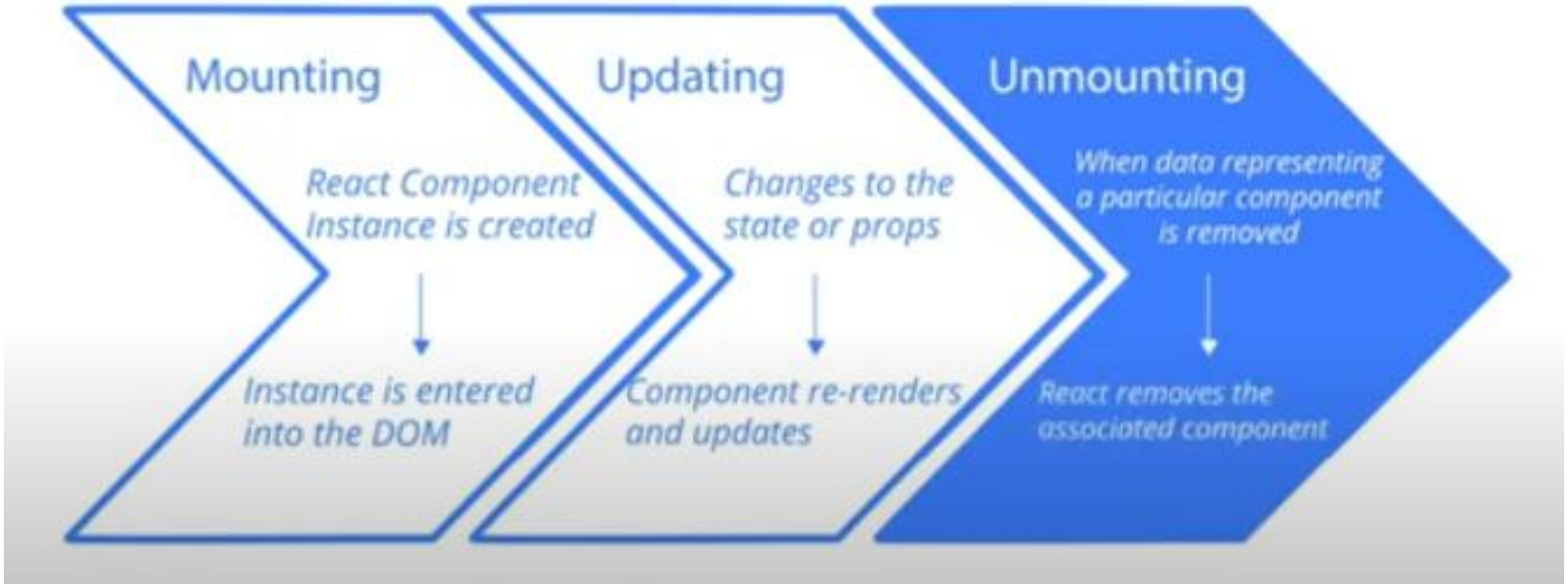


Fruits
Smart Bulb
USB Cable Cord
When 'Nutella' is removed from the list, the list item component representing it is destroyed and unmounted from the DOM.
[Data]
My shopping list

React provides predefined milestones where the developers are allowed to run code.



Class Component Lifecycle



Mounting Phase

- Mounting phase occurs when a React component instance is created and inserted into the DOM.

1. **constructor()** Method
2. **getDerivedStateFromProps()** Method
3. **render()** Method
4. **componentDidMount()** Method

Constructor() method

The constructor is called **before the component is mounted** into the **DOM**. The typical use of a constructor is **to initialize local state variables** and **bind class methods.**

```
class Greeting extends Component {

  constructor(props) {
    super(props);
    this.state = {
      isLoading: false,
      userName: "",
      shoppingList: []
    }
    this.onClickHandler = this.onClickHandler.bind(this);
  }
}
```

Mandatory to call super(props)

The only place where you can initialize state using this.state

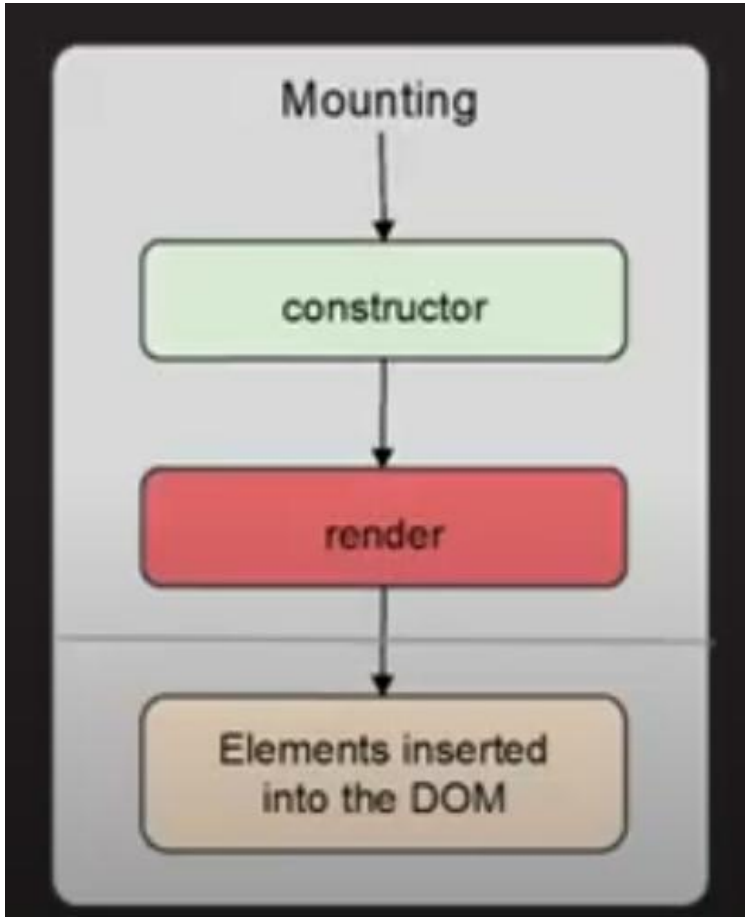
Binding methods to the context of the class

This is also where you initialize state variables by setting this.state to an object. These variables can then be updated using this.setState({}) in class and other lifecycle methods.

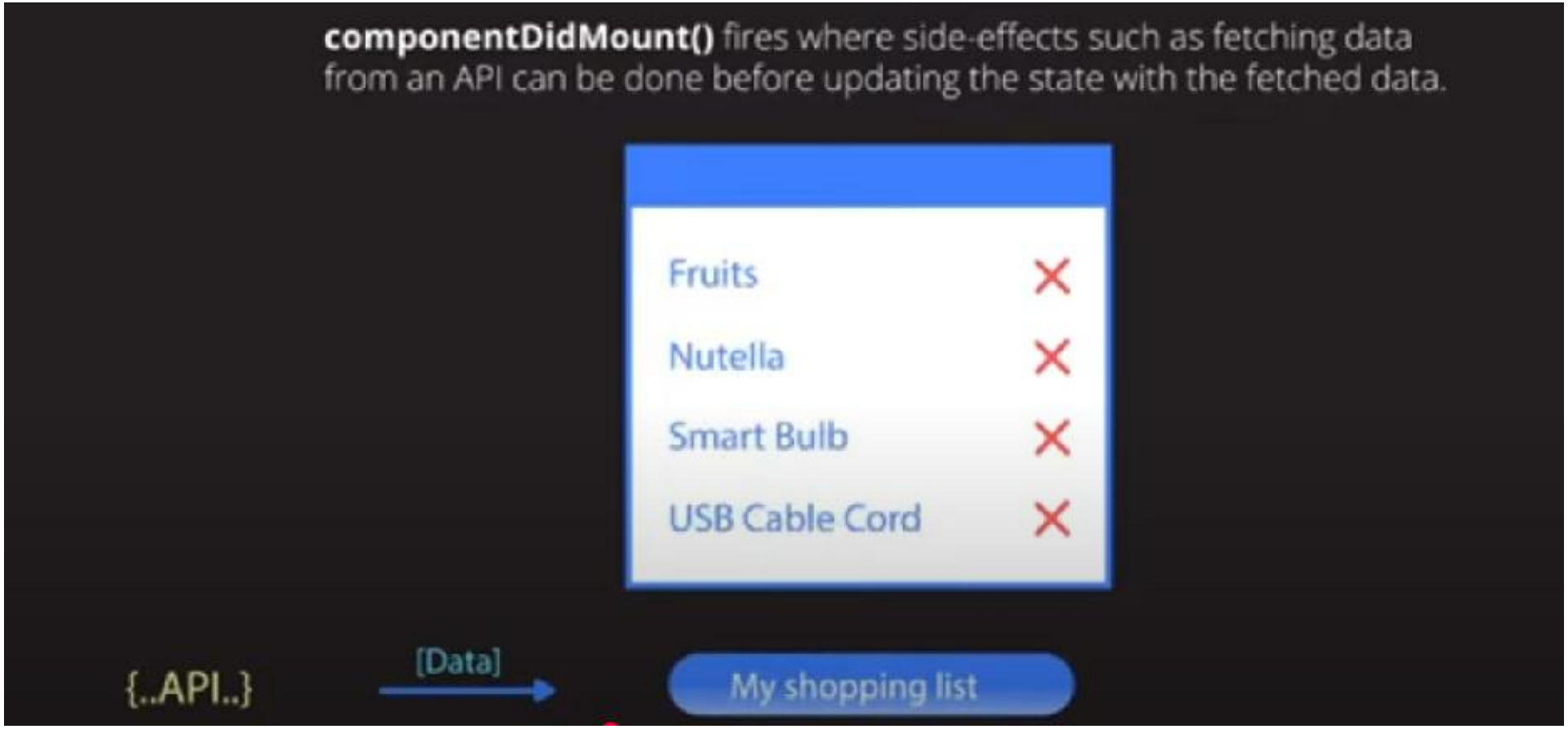
getDerivedStateFromProps() method

This lifecycle hook is used in both the **Mounting and Updating phases** and allows you to **perform state changes** on the basis of data in the props.

render() method



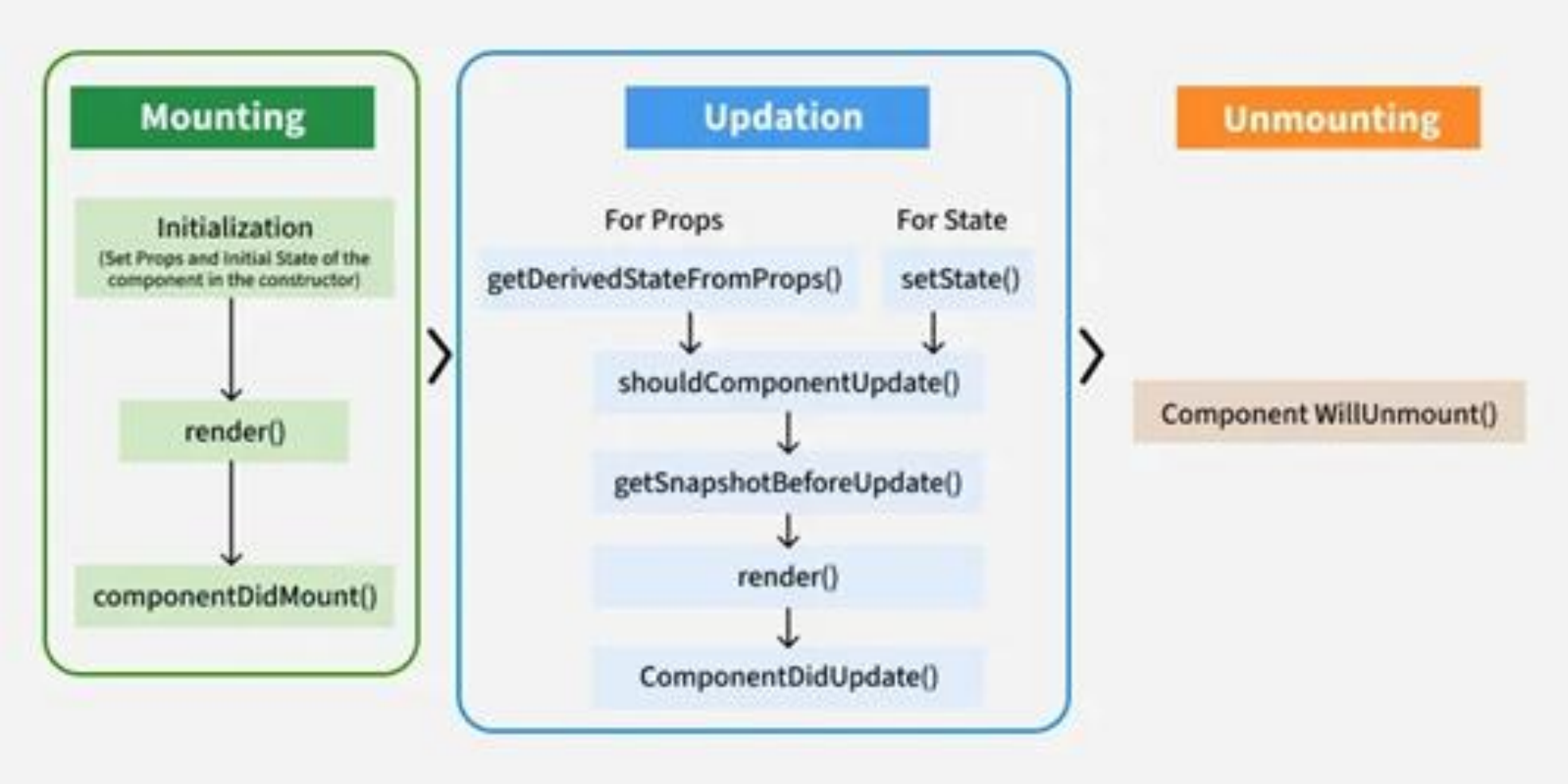
componentDidMount()



Three main phases of the React component lifecycle

- **Mounting:** Initializes, renders, and mounts the component (componentDidMount()).
- **Updating:** Handles state/prop changes, re-renders, and updates (componentDidUpdate()).
- **Unmounting:** Cleans up before removal (componentWillUnmount()).

What is the React Lifecycle?



ssage.tsx × | hello.jsx | header.jsx | funcarrow.jsx | navbar.jsx | # App.css | App.tsx 1 | co

src > components > classcounter.jsx > Classcounter > componentDidMount

```react/classcounter.jsx
import {Component}from "react";
class Classcounter extends Component{
    constructor() {
        super();
        this.state={
            count: 0
        };
        console.log("Component is being Constructed");
    }
    increment=()=> {
        this.setState((prevState)=>({
        count:prevState.count + 1
    }));
};

    componentDidMount()
    {
        console.log("Component is now mounted");

    }

    render()
    {
        console.log("UI is being rendered");
        return(
            <div>
                <p> Count: {this.state.count}</p>
                <button onClick={this.increment}> Click to increment </button>
            </div>
        );
    }
}
export default Classcounter;
```

sage.tsx × | hello.jsx | header.jsx | funcarrow.jsx | navbar.jsx | # App.css | App.tsx 1 ×

src > App.tsx > ...

```
//import Hello from './components/hello';
//import Header from './components/header';
//import NHeader from './components/funcarrow';
//import Navbar from './components/navbar';
//import Counter from './components/counter';
//import Itemclass from './components/itemclass';
import Classcounter from './components/classcounter';


function App() {
  return (
    <div>
      {/*<Header></Header>*/}
      {/*<NHeader></NHeader>*/}
      {/*<Navbar></Navbar>*/}
      {/*<Hello></Hello>*/}
      {/*<Counter></Counter>*/}
      {/*<Itemclass></Itemclass>*/}
      <Classcounter></Classcounter>

        </div>
  );
}

export default App;
```

Vite + React + TS

localhost:5173

Count: 2

Click to increment

Console

top | Filter | Default levels | No Issues | 2 hidden

```
react-dom-client.development.js:24594
Download the React DevTools for a better development experience:
https://react.dev/link/react-devtools
2 Component is being Constructed                    classcounter.jsx:8
2 UI is being rendered                              classcounter.jsx:24
2 Component is now mounted                          classcounter.jsx:18
  UI is being rendered                              classcounter.jsx:24
  UI is being rendered                              classcounter.jsx:24
  UI is being rendered                              classcounter.jsx:24
  UI is being rendered                              classcounter.jsx:24
>
```

src > components > classcounter.jsx > Classcounter > componentDidMount

```
import {Component}from "react";
class Classcounter extends Component{
    constructor() {
        super();
        this.state={
            count: 0
        };
        console.log("Component is being Constructed");
    }
    increment=()=> {
        this.setState((prevState)=>({
        count:prevState.count + 1
    }));
};

    componentDidMount()
    {
        console.log("Component is now mounted");

    }

    shouldComponentUpdate(nextProps, nextState)
    {
        return false;
    }

    render()
    {
        console.log("UI is being rendered");
        return(
            <div>
                <p> Count: {this.state.count}</p>
                <button onClick={this.increment}> Click to increment </button>
            </div>
        );
    }
}
export default Classcounter;
```

Vite + React + TS

localhost:5173

Count: 0

Click to increment

Console

top | Filter | Default levels | No Issues | 2 hidden

react-dom-client.development.js:24594

Download the React DevTools for a better development experience:
https://react.dev/link/react-devtools

Component is being Constructed	classcounter.jsx:8
Component is being Constructed	classcounter.jsx:8
UI is being rendered	classcounter.jsx:29
UI is being rendered	classcounter.jsx:29
Component is now mounted	classcounter.jsx:18
Component is now mounted	classcounter.jsx:18

Props

- Props is short for ‘properties’ and props are a way to pass data from a parent component to a child component in React

Props, short for properties, allow the user to pass arguments or data to components

Properties help make components more dynamic

Props are passed to components in the way similar to that of HTML-tag attributes

Props in a component are **read-only** and cannot be changed

Function props

File Edit Selection View Go Run Terminal Help

react-app

EXPLORER

REACT-APP

- node_modules
  - word-wrap
  - yallist
  - yocto-queue
  - .package-lock.json
- public
- src
  - assets
  - components
    - classcounter.jsx
    - counter.jsx
    - funcarrow.jsx
    - header.jsx
    - hello.jsx
    - itemclass.jsx
    - navbar.jsx
    - Usercard.jsx
  - App.css
  - App.tsx 1
  - index.css
  - main.tsx
  - Message.tsx
  - mycomponent.jsx
  - vite-env.d.ts
- .gitignore
- eslint.config.js
- index.html
- package-lock.json
- package.json
- README.md
- tsconfig.app.json
- tsconfig.json

e.tsx | hello.jsx | header.jsx | funcarrow.jsx | navbar.jsx | App.css | App.tsx 1 | counter.jsx | Usercard.jsx

src > components > Usercard.jsx > Usercard

```jsx
export default function Usercard(props) {
    console.log(props);
    return(
        <div className="border">
            <h2>{props.name}</h2>
            <small> Score: 400</small>
            <p> Welcome to props in react. This is my first learning</p>
        </div>
    )
}
```

PROBLEMS 1 | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

```
Microsoft Windows [Version 10.0.22631.4751]
(c) Microsoft Corporation. All rights reserved.

C:\Users\admin\react-app>npm run dev

> react-app@0.0.0 dev
> vite
```

File Edit Selection View Go Run Terminal Help

react-app

EXPLORER

e.tsx | hello.jsx | header.jsx | funcarrow.jsx | navbar.jsx | App.css | App.tsx 1

src > App.tsx > App

```
REACT-APP
  node_modules
    word-wrap
    yallist
    yocto-queue
  .package-lock.json
  public
  src
    assets
    components
      classcounter.jsx
      counter.jsx
      funcarrow.jsx
      header.jsx
      hello.jsx
      itemclass.jsx
      navbar.jsx
      Usercard.jsx
    App.css
    App.tsx 1
    index.css
    main.tsx
    Message.tsx
    mycomponent.jsx
    vite-env.d.ts
  .gitignore
  eslint.config.js
  index.html
  package-lock.json
  package.json
  README.md
```

```
1  //import Hello from './components/hello';
2  //import Header from './components/header';
3  //import NHeader from './components/funcarrow';
4  //import Navbar from './components/navbar';
5  //import Counter from './components/counter';
6  //import Itemclass from './components/itemclass';
7  //import Classcounter from './components/classcounter';
8  import Usercard from './components/Usercard';
9
10
11 function App() {
12   return (
13     <div>
14       {/*<Header></Header>*/}
15       {/*<NHeader></NHeader>*/}
16       {/*<Navbar></Navbar>*/}
17       {/*<Hello></Hello>*/}
18       {/*<Counter></Counter>*/}
19       {/*<Itemclass></Itemclass>*/}
20       {/*<Classcounter></Classcounter>*/}
21       <Usercard name="Rutuja"></Usercard>
22       <Usercard name="Neha"></Usercard>
23       <Usercard name="Akshata"></Usercard>
24
25         </div>
26   );
27 }
28
29 export default App;
```

Vite + React + TS

localhost:5173

Rutuja

Score: 400

Welcome to props in react. This is my first learning

Neha

Score: 400

Welcome to props in react. This is my first learning

Akshata

Score: 400

Welcome to props in react. This is my first learning

Console

top | Filter | Default levels | No Issues | 10 hidden

```
meant to render a collection of children, use an array instead.
    at throwOnInvalidObjectType (react-dom-client.development.js:4446:13
)
    at reconcileChildFibersImpl (react-dom-client.development.js:5376:11
)
    at react-dom-client.development.js:5415:33
    at reconcileChildren (react-dom-client.development.js:7739:13)
    at beginWork (react-dom-client.development.js:9951:13)
    at runWithFiberInDEV (react-dom-client.development.js:543:16)
    at performUnitOfWork (react-dom-client.development.js:15044:22)
    at workLoopSync (react-dom-client.development.js:14870:41)
    at renderRootSync (react-dom-client.development.js:14850:11)
    at performWorkOnRoot (react-dom-client.development.js:14384:44)
```

```
▶ An error occurred in the <h2> component.                    @react-refresh:267

Consider adding an error boundary to your tree to customize error
handling behavior.
Visit https://react.dev/link/error-boundaries to learn more about error
boundaries.
```

```
▶ {name: 'Rutuja'}    Usercard.jsx?t=1743398833239:18
▶ {name: 'Rutuja'}    Usercard.jsx?t=1743398833239:18
▶ {name: 'Neha'}      Usercard.jsx?t=1743398833239:18
▶ {name: 'Neha'}      Usercard.jsx?t=1743398833239:18
▶ {name: 'Akshata'}   Usercard.jsx?t=1743398833239:18
▶ {name: 'Akshata'}   Usercard.jsx?t=1743398833239:18
▶ {name: 'Rutuja'}    Usercard.jsx?t=1743398845245:18
▶ {name: 'Rutuja'}    Usercard.jsx?t=1743398845245:18
▶ {name: 'Neha'}      Usercard.jsx?t=1743398845245:18
▶ {name: 'Neha'}      Usercard.jsx?t=1743398845245:18
▶ {name: 'Akshata'}   Usercard.jsx?t=1743398845245:18
▶ {name: 'Akshata'}   Usercard.jsx?t=1743398845245:18
▶ {name: 'Rutuja'}    Usercard.jsx:2
▶ {name: 'Rutuja'}    Usercard.jsx:2
▶ {name: 'Neha'}      Usercard.jsx:2
▶ {name: 'Neha'}      Usercard.jsx:2
▶ {name: 'Akshata'}   Usercard.jsx:2
▶ {name: 'Akshata'}   Usercard.jsx:2
▶ {name: 'Rutuja'}    Usercard.jsx:2
▶ {name: 'Rutuja'}    Usercard.jsx:2
▶ {name: 'Neha'}      Usercard.jsx:2
▶ {name: 'Neha'}      Usercard.jsx:2
▶ {name: 'Akshata'}   Usercard.jsx:2
▶ {name: 'Akshata'}   Usercard.jsx:2
```

File Edit Selection View Go Run Terminal Help react-app

EXPLORER

REACT-APP

- node_modules
  - word-wrap
  - yallist
  - yocto-queue
  - .package-lock.json
- public
- src
  - assets
  - components
    - classcounter.jsx
    - counter.jsx
    - funcarrow.jsx
    - header.jsx
    - hello.jsx
    - itemclass.jsx
    - navbar.jsx
    - Usercard.jsx
  - App.css
  - App.tsx 1
  - index.css
  - main.tsx
  - Message.tsx
  - mycomponent.jsx
  - vite-env.d.ts
- .gitignore
- eslint.config.js
- index.html
- package-lock.json

e.tsx | hello.jsx | header.jsx | funcarrow.jsx | navbar.jsx | App.css | App.tsx 1 | counter.jsx | Usercard.jsx

src > components > Usercard.jsx > Usercard

```jsx
export default function Usercard(props) {
    console.log(props);
    return(
        <div className="border">
            <h2>{props.name}</h2>
            <small> Score:{props.score}</small>
            <p> Welcome to props in react. This is my first learning</p>
        </div>
    )
}
```

File Edit Selection View Go Run Terminal Help

react-app

EXPLORER

REACT-APP

- node_modules
  - word-wrap
  - yallist
  - yocto-queue
  - .package-lock.json
- public
- src
  - assets
  - components
    - classcounter.jsx
    - counter.jsx
    - funcarrow.jsx
    - header.jsx
    - hello.jsx
    - itemclass.jsx
    - navbar.jsx
    - Usercard.jsx
  - App.css
  - App.tsx 1
  - index.css
  - main.tsx
  - Message.tsx
  - mycomponent.jsx
  - vite-env.d.ts
- .gitignore
- eslint.config.js
- index.html
- package-lock.json
- package.json
- README.md
- tsconfig.app.json

e.tsx | hello.jsx | header.jsx | funcarrow.jsx | navbar.jsx | App.css | App.tsx 1 | counter.jsx | Usercar

src > App.tsx > default

```
//import Hello from './components/hello';
//import Header from './components/header';
//import NHeader from './components/funcarrow';
//import Navbar from './components/navbar';
//import Counter from './components/counter';
//import Itemclass from './components/itemclass';
//import Classcounter from './components/classcounter';
import Usercard from './components/Usercard';


function App() {
  return (
    <div>
      {/*<Header></Header>*/}
      {/*<NHeader></NHeader>*/}
      {/*<Navbar></Navbar>*/}
      {/*<Hello></Hello>*/}
      {/*<Counter></Counter>*/}
      {/*<Itemclass></Itemclass>*/}
      {/*<Classcounter></Classcounter>*/}
      <Usercard name="Rutuja" score="80">
        <p> User description</p>
      </Usercard>
      <Usercard name="Neha" score="70">
        <div>
          <button> Click Here </button>
        </div>
      </Usercard>
      <Usercard name="Akshata" score="50"></Usercard>

        </div>
  );
}

export default App;
```

Vite + React + TS

localhost:5173

Rutuja

Score:80

Welcome to props in react. This is my first learning

Neha

Score:70

Welcome to props in react. This is my first learning

Akshata

Score:50

Welcome to props in react. This is my first learning

Console

top | Filter | Default levels | No Issues | 2 hidden

react-dom-client.development.js:24594

Download the React DevTools for a better development experience:
https://react.dev/link/react-devtools

```
{name: 'Rutuja', score: '80'}        Usercard.jsx:
{name: 'Rutuja', score: '80'}        Usercard.jsx:
{name: 'Neha', score: '70'}          Usercard.jsx:
{name: 'Neha', score: '70'}          Usercard.jsx:
{name: 'Akshata', score: '50'}       Usercard.jsx:
{name: 'Akshata', score: '50'}       Usercard.jsx:
```

File Edit Selection View Go Run Terminal Help

react-app

EXPLORER

REACT-APP

- node_modules
  - word-wrap
  - yallist
  - yocto-queue
  - .package-lock.json
- public
- src
  - assets
  - components
    - classcounter.jsx
    - counter.jsx
    - funcarrow.jsx
    - header.jsx
    - hello.jsx
    - itemclass.jsx
    - navbar.jsx
    - Usercard.jsx
  - App.css
  - App.tsx 1
  - index.css
  - main.tsx
  - Message.tsx
  - mycomponent.jsx
  - vite-env.d.ts
- .gitignore
- eslint.config.js
- index.html

e.tsx | hello.jsx | header.jsx | funcarrow.jsx | navbar.jsx | App.css | App.tsx 1 | counter.jsx | Usercard.jsx

src > components > Usercard.jsx > Usercard

```jsx
export default function Usercard(props) {
    console.log(props);
    return(
        <div className="border">
            <h2>{props.name}</h2>
            <small> Score:{props.score}</small>
            {props.children}
            <p> Welcome to props in react</p>
        </div>
    )
}
```

File Edit Selection View Go Run Terminal Help

react-app

EXPLORER

REACT-APP

- node_modules
  - word-wrap
  - yallist
  - yocto-queue
  - .package-lock.json
- public
- src
  - assets
  - components
    - classcounter.jsx
    - counter.jsx
    - funcarrow.jsx
    - header.jsx
    - hello.jsx
    - itemclass.jsx
    - navbar.jsx
    - Usercard.jsx
  - App.css
  - App.tsx 1
  - index.css
  - main.tsx
  - Message.tsx
  - mycomponent.jsx
  - vite-env.d.ts
- .gitignore
- eslint.config.js
- index.html
- package-lock.json
- package.json
- README.md
- tsconfig.app.json

e.tsx | hello.jsx | header.jsx | funcarrow.jsx | navbar.jsx | App.css | App.tsx 1 | counter.jsx | Usercar

src > App.tsx > default

```
//import Hello from './components/hello';
//import Header from './components/header';
//import NHeader from './components/funcarrow';
//import Navbar from './components/navbar';
//import Counter from './components/counter';
//import Itemclass from './components/itemclass';
//import Classcounter from './components/classcounter';
import Usercard from './components/Usercard';


function App() {
  return (
    <div>
      {/*<Header></Header>*/}
      {/*<NHeader></NHeader>*/}
      {/*<Navbar></Navbar>*/}
      {/*<Hello></Hello>*/}
      {/*<Counter></Counter>*/}
      {/*<Itemclass></Itemclass>*/}
      {/*<Classcounter></Classcounter>*/}
      <Usercard name="Rutuja" score="80">
        <p> User description</p>
      </Usercard>
      <Usercard name="Neha" score="70">
        <div>
          <button> Click Here </button>
        </div>
      </Usercard>
      <Usercard name="Akshata" score="50"></Usercard>

        </div>
  );
}

export default App;
```

Vite + React + TS

localhost:5173

Rutuja

Score:80

User description

Welcome to props in react

Neha

Score:70

Click Here

Welcome to props in react

Akshata

Score:50

Welcome to props in react

Console

top Filter Default levels No Issues 2 hidden

react-dom-client.development.js:24594

Download the React DevTools for a better development experience:
https://react.dev/link/react-devtools

```
{name: 'Rutuja', score: '80', children: {…}}   Usercard.jsx:2
{name: 'Rutuja', score: '80', children: {…}}   Usercard.jsx:2
{name: 'Neha', score: '70', children: {…}}     Usercard.jsx:2
{name: 'Neha', score: '70', children: {…}}     Usercard.jsx:2
{name: 'Akshata', score: '50'}                 Usercard.jsx:2
{name: 'Akshata', score: '50'}                 Usercard.jsx:2
```

Class Props

Student need to try above same example using class

Props vs State

Sr. No	Props	State
1.	Props are passed from parent to a child component	State is managed inside the component
2.	Props are immutable, it should not be modified in child component	State are changed using hooks like ‘useState()’

Event Handling

Introduction

- React.js provides a straightforward way to handle user events, such as clicks, form submissions, and key presses.
- Event handling in React.js follows a similar pattern to traditional JavaScript event handling, but with a few key differences.
- In React.js, event handlers are defined as methods on the component class or as functions in functional components.

Introduction

- React.js also provides a set of synthetic events, which are cross-browser wrappers around the native browser events. These synthetic events provide a consistent interface for event handling, making it easier to write code that works across different browsers and environments.

Points to remember

- React events are named using camelCase, rather than lowercase.
- With JSX you pass a function as the event handler, rather than a string.

Example:

- <button onClick={activateLasers}> Activate Lasers </button>

Points to remember

- Another difference is that you cannot return false to prevent default behavior in React.
- You must call preventDefault explicitly. For example, with plain HTML, to prevent the default form behavior of submitting, you can write:

Points to remember

```
<form onsubmit="console.log('You clicked submit.'); return false">
  <button type="submit">Submit</button>
</form>
```

In React, this could instead be:

```
function Form() {
  function handleSubmit(e) {
    e.preventDefault();
    console.log('You clicked submit.');
  }

  return (
    <form onSubmit={handleSubmit}>
      <button type="submit">Submit</button>
    </form>
  );
}
```

Example

```
class Toggle extends React.Component {
  constructor(props) {
    super(props);
    this.state = {isToggleOn: true};

    // This binding is necessary to make `this` work in the callback
    this.handleClick = this.handleClick.bind(this);
  }

  handleClick() {
    this.setState(prevState => ({
      isToggleOn: !prevState.isToggleOn
    }));
  }

  render() {
    return (
      <button onClick={this.handleClick}>
        {this.state.isToggleOn ? 'ON' : 'OFF'}
      </button>
    );
  }
}
```

Instead of using class fields syntax, you can use an arrow function in the callback:

```
class LoggingButton extends React.Component {
  handleClick() {
    console.log('this is:', this);
  }

  render() {
    // This syntax ensures `this` is bound within handleClick
    return (
      <button onClick={() => this.handleClick()}>
        Click me
      </button>
    );
  }
}
```

Passing Arguments to Event Handlers

```
<button onClick={(e) => this.deleteRow(id, e)}>Delete Row</button>
<button onClick={this.deleteRow.bind(this, id)}>Delete Row</button>
```

Handling Form Submissions

```
import React, { useState } from 'react';

function Form() {
  const [inputValue, setInputValue] = useState('');

  const handleSubmit = (event) => {
    event.preventDefault();
    console.log('Submitted value:', inputValue);
  };

  const handleInputChange = (event) => {
    setInputValue(event.target.value);
  };
```

- return (
- <form onSubmit={handleSubmit}>
- <input type="text" value={inputValue} onChange={handleInputChange} />
- <button type="submit">Submit</button> </form>
-); }

Handling keyboard events

```
import React, { useState } from 'react';

function Input() {
  const [inputValue, setInputValue] = useState('');

  const handleKeyDown = (event) => {
    if (event.key === 'Enter') {
      console.log('Submitted value:', inputValue);
    }
  };
```

- const handleInputChange = (event) => {
  setInputValue(event.target.value);
- };
- return (<input type="text" value={inputValue}
  onKeyDown={handleKeyDown} onChange={handleInputChange} />
-);
- }

Conditional Rendering

- Your components will often need to display different things depending on different conditions. In React, you can conditionally render JSX using JavaScript syntax like if statements, &&, and ? : operators.

```
import React, { useState } from "react";

const App = () => {
  const [isLoggedIn, setIsLoggedIn] = useState(false);

  const handleLogin = () => {
    setIsLoggedIn(true);
  };

  const handleLogout = () => {
    setIsLoggedIn(false);
  };

  return (
    <div>
      {isLoggedIn ? (
        <div>
          <h1>Welcome Back!</h1>
          <button onClick={handleLogout}>Logout</button>
        </div>
      ) : (
        <div>
          <h1>Please Login</h1>
          <button onClick={handleLogin}>Login</button>
        </div>
      )}
    </div>
  );
};

export default App;
```

Forms

Forms in React

- Form tags
- How to get values from form fields
- Form methods
- useRef

React Routing

React Router Dom

- Router
- Routes/Switch
- Route
- Link tag
- useHistory/ useNavigate

References

- https://react.dev/
- https://www.w3schools.com/react/react_components.asp
- https://legacy.reactjs.org/
- https://www.geeksforgeeks.org/what-are-the-differences-between-npm-and-npx/
- https://reactrouter.com/en/main/routers/create-browser-router